

Method Selection and Planning

CO3 G10

Will Barlow
Harris Byrom
Corbin Hedger
Dmytro Kovalenko
Sharjil Munshi
Oliver Spence
Joshua Thomas
Leon Varghese

Part A - Software Engineering Methods

At the beginning of the project, research was done into project management tools. It was important this was done early as the more progress that is made the harder it is to start using a tool. So if we were to use something it should be right from the start. Please see some research notes below:

Researching and selecting a task-management service:

1) Jira:

Very advanced and perfect for professional and complex projects.

Great for agile development paradigm and for structured large teams.

Might be a bit overwhelming for some and has a bit of a learning curve.

2) Trello:

Simple, visual task management tool which lets you select priorities, drop down tasks and change their progress status (todo -> in progress -> done).

You can use it for everything - bugs, issues, ideas, plans, checklists.

Very intuitive and almost no learning curve.

3) Github issues:

Built in issue tracking in github.

Good for managing tasks and issues directly tied to the code.

Good only for code-focused management, rather than general task tracking.

Despite these options being available, we decided instead to manage task tracking and assignment using a Google Docs meeting log. In this log we recorded our group meetings, who was present, what was completed in the meeting, and tasks to be completed for the next meeting and by who. Due to the small scale of the project we felt the overhead of using one of the task management tools researched above was too great. A simple, easy to understand, meeting log presents no learning curve for any team members as we are all familiar with Google Docs. Furthermore it provided a versioned environment as required by the project.

Throughout the project, we have had no major issues with task tracking and assignment. We discussed tasks that needed to be completed in our group meetings and assigned them across the team. Then the outcome was recorded in the meeting log. This has proved more than suitable for our needs. However, we can see the value of using a dedicated task management tool for larger projects and/or larger teams, where it is more difficult to have semi-informal discussions on tasks to be completed with the whole group.

For all project documents, we used a shared Google Drive folder. This gave us a versioned environment to collaborate on documents such as deliverables, research notes, planning documents, and risk assessments. All documents created on the Google Drive platform are versioned by default as required by the project. Google Drive is a tool that all team members have experience with so there was no learning overhead for anyone, making it very suitable for our needs.

As well as researching project management tools, research was also carried out into game engines and libraries that could be used in the implementation. Game engines considered

include: LibGDX, jMonkeyEngine, and Litiengine. The decision was made to use LibGDX for the project. Firstly, it is very mature, has lots of documentation, and lots of example projects. This means that the overhead of learning the engine is less than the other options. Secondly, some team members already had experience with it.

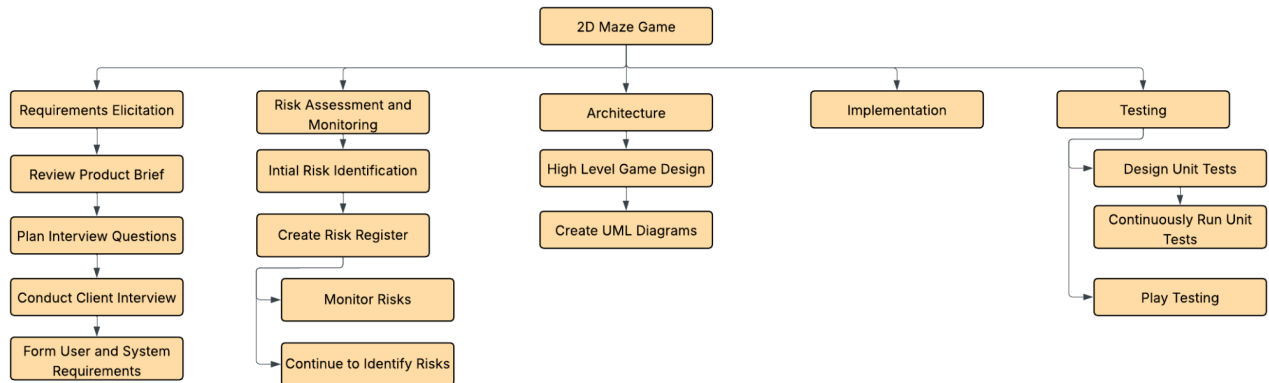
Research was also carried out into Github, a Git repository hosting site. Allowing easy collaboration using Git. A combination of Git and Github provides a highly versioned environment, pull request workflows, an ability to raise issues with Github Issues, and an easy way to rollback any commits containing bugs. For these reasons, combined with high familiarity among team members, lead us to choose this to manage our codebase.

Part B - Approach to Team Organisation

The team's approach to organisation started off with creating a plan, with the client interview being one of the first on our priority list. We all made sure to focus on creating questions. We also created a work breakdown diagram. Creating this diagram allowed us to see what tasks needed to be done. Our group then decided to split into roughly two halves. One half focused on implementation and coding, whereas the other half focused on documentation. The documentation side then split further based on our decomposition diagram to focus on different tasks which we did to increase efficiency. For the documentation team, some focused on method selection and planning, while others focused on requirements and architecture. We made sure to follow our work breakdown diagram, but made changes to it where our requirements changed. Completing the deliverables for each of the tasks was the final stage of each problem that each sub team worked on. Organising our work and tasks this way was appropriate for this project as the problem that we were looking to solve is able to be decomposed into smaller problems, allowing sub teams to each work on the different parts simultaneously - hence increasing efficiency and productivity. The work breakdown diagram also helped to guide us on sub teams to create and what tasks to allocate to who.

Part C - Project Plan

Initial Work Breakdown



Project Schedule

ID	Description	Estimated start and end week	Dependencies	People
T1.1	Review product brief	2-2	None	Everyone
T1.2	Draft interview questions	2-3	T1.1	WB, HB, JT, SM, CH, OS
T1.3	Finalise and sort interview questions.	3	T1.2	WB, HB, JT, SM, CH
T1.4	Conduct client interview	3	T1.3	WB, HB, JT, SM, CH, LV
T1.5	Transcript Interview	3	T1.4	WB, HB, SM
T1.6	Form user requirements	3-4	T1.5	CH, JT, SM, WB, HB
T1.7	Form non functional & functional system requirements	3-4	T1.6	CH, JT, SM, WB, HB
T1.8	Form requirements deliverable	3-5	T1.7	JT
T2.1	Initial risk assessment	2	None	WB, LV, HB
T2.2	Design risk register	2	None	WB, HB
T2.3	Monitor risks	All	T2.1, T2.2	Everyone

T2.4	Form risk assessment deliverable	3-5	T2.1, T2.2	WB
T3.1	High level game design	4-5	T1.7	DK
T3.2	Create UML diagrams	4-5	T3.1	HB, CH
T3.3	Form architecture deliverable	4-5	T3.2	HB, CH, JT
T4.1	Implementation	5-6	T3.2	DK, SM, OS, CH
T4.2	Form implementation deliverable	6	T3.2	SM
T5.1	Design unit tests	5-6	T1.7	DK, SM, OS, CH
T5.2	Run unit tests	5-6	T4.1, T5.1	DK, SM, OS, CH
T5.3	Play testing	5-6	T4.1	DK, SM, OS, CH